

Slide notes — Member 5 — ISSD

The main talk is slides 1–8; slides 9–12 are supporting evidence/Q&A. No fixed speaking length is imposed. Use LIVE_DEMO.pdf for exact terminal commands.

Live transitions: **slide 3** → **Part 1**, **slide 5** → **Part 2**, **slide 7** → **Part 3**. Finish with **Part 4 cleanup**. Recorded findings are not predetermined live output.

Slide 1 — Introduce the race before the demonstration

Introduce Member 5 and the SEED Race-Condition Lab. A race condition means the outcome depends on the ordering of concurrent operations. Our case is time of check to time of use: a program checks one target and later opens another target through the same name. This is a local application-level lab, not Dirty COW or a remote attack. The actual terminal demonstration comes after the mechanism and the audience cues.

Slide 2 — Explain real versus effective identity

Point to the two actual operations. access uses the real user identity. fopen resolves the name again and uses the process credentials for the open. In our deliberately configured Set-UID victim the effective identity is root. The attacker changes the symlink from writable /dev/null to protected /etc/passwd in the interval. A symlink does not grant privileges by itself. Do not imply that every Ubuntu installation has this program or that file permissions disappeared. Source: vulp.c, S03/S04, and the SEED task PDF.

Slide 3 — Switch to LIVE_DEMO Part 1

Use LIVE_DEMO Part 0 before class, then Part 1 in its exact A/B order. A initializes the /dev/null link. B runs vulp_slow. Only after Check passed, switch A to /etc/passwd during the ten-second wait. Wait for the victim to return, inspect the full record, then run non-sudo su - test and id. Password input is interactive. Explain that this is the official slow-machine simulation, not the no-delay result. Exit the root shell and reset before the next ordinary-user trial. Return to slide 4. If timing misses, say so and use actual saved S06/S07 as recorded evidence.

Slide 4 — Explain why the observed output proves the gain

The first identity output shows seed. The exact-record check establishes that the complete 43-character entry, with seven fields and UID/GID zero, exists once. The meaningful final proof is an actual non-sudo su login followed by id. whoami may say root because both account names map to UID zero. A hash change, a program exit status zero, or an account name alone is insufficient. In Task 1 we inserted the record administratively only to validate it; in the actual attack the privileged victim performed the append. Our saved results S07, S09 and S11 preserve that distinction.

Slide 5 — Switch to the genuine no-delay atomic demonstration

These are historical measurements, rounded to three decimals on the slide; the report contains exact values and labels. They do not promise that the next trial will finish in one attempt or that one method is always faster. The supplied C victim has no artificial sleep. If presenting live, use Part 2: B resets and creates a unique shared label, A starts the bounded atomic attacker, and B starts the 30-second no-delay monitor only after initialization. Inspect any changed file and verify non-sudo login/id; then exit root and reset. If the 30 seconds expire unchanged, state that the live attempt did not win and use the labelled actual S09/S11 evidence. Do not call that short class attempt the original 300-second experiment. Return to slide 6.

Slide 6 — Explain the attacker's own race and its repair

The unlink/create sequence is not atomic. If the victim already passed access and reaches fopen while the name is absent, a+ can create a root-owned regular file. The attacker can then encounter File exists and later an unlink permission error. The sticky directory rule concerns deletion of an entry; group write permission on the file does not grant that deletion. In the old partial log, repeated successful victim exits did not prove an append to /etc/passwd. We preserved the file, metadata and logs before administrator-assisted reset. The improved attacker creates two links first and atomically exchanges them with RENAME_EXCHANGE; initialize fully before the victim starts. This fixes the attacker's gap, while the victim still checks and opens separately.

Slide 7 — Explain both controls; optionally show the stable-link denial live

For the first experiment we changed the program and left both OS controls disabled. seteuid(getuid()) drops effective privilege before access and fopen and keeps it dropped through writing/closing. The allowed-file control confirmed normal functionality. For the second we returned to the original vulnerable victim and enabled only protected_symlinks, leaving protected_regular at zero. The kernel blocks a root-effective follower of a seed-owned link under sticky root-owned /tmp in this case. In Part 3 of the live guide, a stable /dev/null link should produce Open failed: Permission denied even though the real-user check can pass. That is a short explanatory control, not a live repetition of five minutes. Finite negative trials alone are not universal proof. These two mechanisms address different layers.

Slide 8 — Close the main presentation and perform cleanup

The gain was a verified local UID-zero login, not merely a changed text file. Correct account-database handling supports authentication integrity. Emphasize secure descriptor-based handling, appropriate permissions/trusted directories, safe temporary creation and only the required privilege. A security-relevant atomic operation is useful; merely making the attacker faster does not secure the victim. OS symlink protection is defence in depth and context-specific. Use Part 4 after class or rehearsal: the supplied cleanup restores the original password file, removes the test paths and Set-UID, and applies the explicitly chosen 1/2 runtime policy. The saved 0/0 file is not asserted to be original defaults. Stop the main talk here; the remaining slides are supporting material.

Slide 9 — Evidence backup

Use only as recorded evidence, not as a substitute disguised as a live result. These are labelled readability crops from S09, task2b-audit1-20261003-064223. The complete original screenshot and full terminal/record checks are in the evidence package. The current live run may have different output or counts. The fragments are not a newly manufactured terminal screen.

Slide 10 — Measurement and failure Q&A

The original log stopped without a final summary, so we do not invent its final totals. The old XYZ file and original logs were preserved before reset. Hash change requires record/login verification, and exit status zero can just mean an append to /dev/null or the wrong temporary file. The wall times include orchestration overhead; their small values do not establish a general success probability or throughput comparison. Monitor and screenshot activity affects scheduling.

Slide 11 — Recovery instructions

Keep LIVE_DEMO.pdf available off the projected terminals. Follow its reset and cleanup sequence. A says Attacker, B says Victim and results; both start as seed. The same run label is passed via class-demo-label.txt. Attack initialization must appear before starting the victim. The supplied runner is bounded and

stops the attacker when the monitor ends. If any setup helper reports an error, address that actual error before continuing. Authentication is interactive; no password is embedded in any script.

Slide 12 — Attribution and completion status

The teaching content and vulnerable-program pattern are attributed to Wenliang Du / SEED Labs under CC BY-NC-SA 4.0. See the report for full references, source modifications and assistance disclosure. Actual commands/evidence were collected in the user's VM; authentication input remained interactive. The practical and documents are complete, but a future oral presentation/rehearsal or submission upload is not claimed to have already occurred. Unknown personal/admin fields were omitted at the user's request.